Question 2: Spatial Convolution Implementation

Introduction

This report describes the implementation of digital image processing algorithms in the spatial domain, developed entirely from scratch in C, without relying on external computer vision libraries. The image format chosen for testing was PGM (*Portable Gray Map*).

Implementation Methodology

Reading and writing PGM images were handled by directly parsing the files at both byte and text levels (depending on the file's *magic number*). To optimize CPU cache memory access, image pixels were allocated sequentially in a single one-dimensional array. A mathematical flattening formula (`index = y * width + x`) was utilized to simulate 2D matrix coordinate access.

Convolution Algorithm

The spatial convolution was implemented by encapsulating the new pixel calculation within a modular function. To prevent segmentation faults and memory boundary violations—which occur when the neighborhood window extends beyond the image limits—we strictly followed the geometric restriction outlined in the project specification.

Let d be the size of the square *kernel* (which must have odd dimensions). The algorithm explicitly ignores the first and last $k = \left\lfloor \frac{d}{2} \right\rfloor$ rows and columns. Two nested `for` loops define this valid "safe zone" for processing. The ignored boundary pixels are left completely unprocessed, retaining their initial zero (black) values provided by the `calloc()` memory allocation.

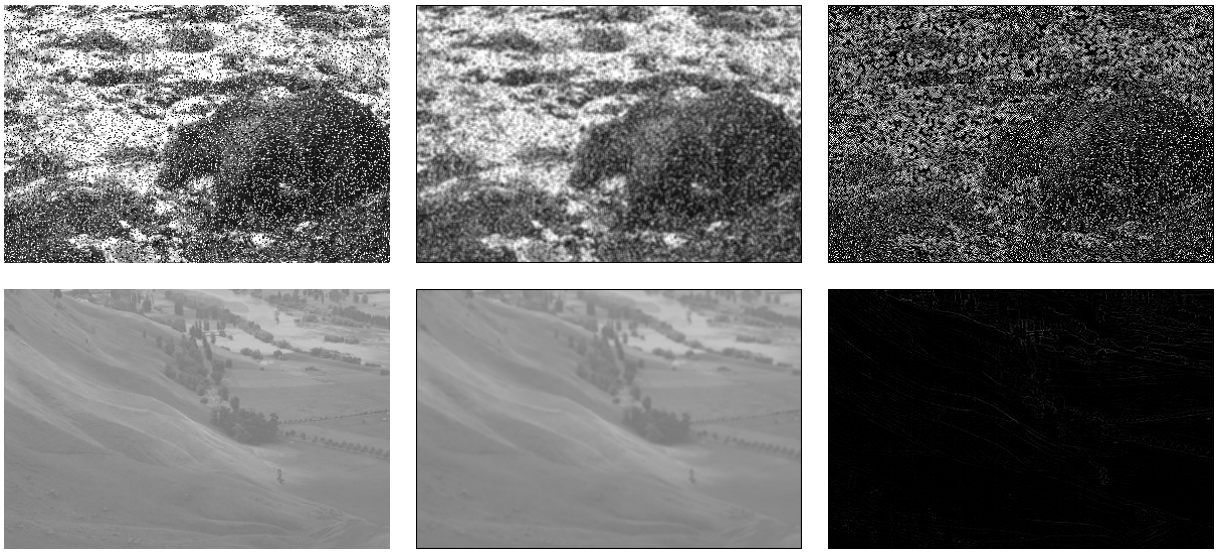
```c
unsigned char aplicar_convolucao_pixel(Image img, int x, int y, int kernel_size, int
*kernel, int soma_pesos_kernel)
{
    int k = kernel_size / 2;
    int soma_convolucao = 0;
    for (int i = -k; i <= k; i++)
    {
        for (int j = -k; j <= k; j++)
        {
            int img_y = y + i;
            int img_x = x + j;
            int indice_img = img_y * img.width + img_x;
            int kern_y = i + k;
            int kern_x = j + k;
            int indice_kernel = kern_y * kernel_size + kern_x;
            soma_convolucao += img.pixels[indice_img] * kernel[indice_kernel];
        }
    }
    int valor_final = soma_convolucao / soma_pesos_kernel;
    // Clamping: garante que o valor fique entre 0 e 255
    if (valor_final > 255)
        valor_final = 255;
    if (valor_final < 0)
        valor_final = 0;
    return (unsigned char)valor_final;
}
```

Results: Low-Pass and High-Pass Filters

The algorithms were tested on the sample image provided in the virtual classroom. Two distinct convolution filters were applied using 3×3 kernels:

1. **Low-Pass Filter (Mean Filter):** This *kernel* assigns a weight of 1 to all positions, requiring a normalization factor of 9. The theoretical expectation is a smoothed (blurred) image, which effectively reduces high-frequency noise at the cost of edge sharpness.
2. **High-Pass Filter (Laplacian):** This *kernel* is designed to highlight abrupt intensity transitions. The expected result is a predominantly dark image where only the edges and contours of the original objects are illuminated. In this specific implementation, negative convolution results were clamped to zero (black).

The visual comparison of the obtained results is presented below.



Original Image | Low-Pass Filter (Mean) | High-Pass Filter (Laplacian)

Conclusion

The implementation successfully applied convolution masks in the spatial domain, covering both High-Pass and Low-Pass filters. The Low-Pass filter proved effective at smoothing images by averaging neighboring pixel values, reducing noise at the cost of some detail. The High-Pass filter, on the other hand, enhanced edges and fine details by means of the Laplacian mask. However, a key limitation observed was its high sensitivity to salt and pepper noise, since the filter amplifies extreme pixel values across the image, which can degrade the output significantly. This highlights the importance of pre-processing steps, such as noise removal, before applying High-Pass filters in practice.